

Diagram 1 — Entity-Relationship (ER) Diagram

How to draw in draw.io

Use **rectangle entities** with two sections (header + attributes). Connect them with **crow's foot notation** lines.

Entities & their attributes

DRIVER

Attribute	Type	Key
_id	ObjectId	PK
name	String	
email	String	
phone	String	
password	String	
licenseNumber	String	
vehicleNumber	String	
role	String ("driver" or "regulator")	
isVerified	Boolean	
qrToken	String	
qrCode	String	
ratePerKm	Number	
area	String	
location_lat	Number	
location_lng	Number	
location_updatedAt	Date	
createdAt / updatedAt	Date	

TRIP

Attribute	Type	Key
_id	ObjectId	PK
driver	ObjectId	FK → DRIVER
customerName	String	
startLocation	String	

Attribute	Type	Key
endLocation	String	
distanceKm	Number	
ratePerKm	Number	
totalFare	Number	
startTime	Date	
endTime	Date	
status	String (pending / ongoing / completed / cancelled)	
syncedToCloud	Boolean	

RIDE_REQUEST

Attribute	Type	Key
_id	ObjectId	PK
driver	ObjectId	FK → DRIVER
trip	ObjectId	FK → TRIP
customerName	String	
pickupLat / pickupLng	Number	
pickupAddress	String	
destLat / destLng	Number	
destAddress	String	
estimatedDistanceKm	Number	
driverRatePerKm	Number	
suggestedRatePerKm	Number	
agreedRatePerKm	Number	
status	String (pending / accepted / rejected / expired)	
syncedToCloud	Boolean	

RECEIPT

Attribute	Type	Key
_id	ObjectId	PK
trip	ObjectId	FK → TRIP
driver	ObjectId	FK → DRIVER

Attribute	Type	Key
receiptNumber	String	
customerName	String	
distanceKm	Number	
ratePerKm	Number	
totalFare	Number	
issuedAt	Date	

Relationships (crow's foot)

From	To	Label	Notation
DRIVER	TRIP	completes	one-to-many (one driver, many trips)
DRIVER	RIDE_REQUEST	receives	one-to-many
DRIVER	RECEIPT	issues	one-to-many
TRIP	RIDE_REQUEST	created from	one-to-zero-or-one (a trip may come from a ride request)
TRIP	RECEIPT	generates	one-to-one (exactly one receipt per completed trip)

Diagram 2 — Class Diagram

How to draw in draw.io

Use **UML class boxes** (three sections: name / attributes / methods). Use **solid arrows** for associations, **dashed arrows** for dependencies.

Classes

Models (data layer)

Class	Key attributes	Methods
Driver	_id, name, email, phone, password, licenseNumber, vehicleNumber, role, isVerified, qrToken, qrCode, ratePerKm, area, location	+comparePassword(candidate): Boolean
Trip	_id, driver, customerName, startLocation, endLocation, distanceKm, ratePerKm, totalFare, startTime, endTime, status, syncedToCloud	—

Class	Key attributes	Methods
RideRequest	_id, driver, trip, customerName, pickup coords/address, dest coords/address, estimatedDistanceKm, driverRatePerKm, suggestedRatePerKm, agreedRatePerKm, status, syncedToCloud	—
Receipt	_id, trip, driver, receiptNumber, customerName, distanceKm, ratePerKm, totalFare, issuedAt	—

Controllers (business logic layer)

Class	Methods
AuthController	+register(), +login(), +phoneLogin(), -signToken()
DriverController	+getDriverProfile(), +updateQRCode(), +getNearbyDrivers(), +getDriverByQR(), +updateLocation()
TripController	+createTrip(), +endTrip(), +getMyTrips(), +getIncome()
RideRequestController	+createRideRequest(), +getIncomingRequests(), +getRequestStatus(), +respondToRequest()
RateController	+updateRate(), +getAreaRates()
AdminController	+listDrivers(), +setDriverVerification(), +getAllTrips(), +getStats()

Middleware

Class	Methods
AuthMiddleware	+protect(), +requireRole(role): Function

Relationships

Solid arrows (association — model to model):

- Driver 1 → 0..* Trip
- Driver 1 → 0..* RideRequest
- Driver 1 → 0..* Receipt
- Trip 1 → 0..1 RideRequest
- Trip 1 → 1 Receipt

Dashed arrows (dependency — controller uses model):

- AuthController - - ► Driver
- DriverController - - ► Driver
- TripController - - ► Trip, Receipt
- RideRequestController - - ► RideRequest, Driver, Trip, Receipt
- RateController - - ► Driver

- AdminController - - ► Driver, Trip, RideRequest

Dashed arrows (middleware guards controllers):

- AuthMiddleware - - ► AuthController, DriverController, TripController, RideRequestController, RateController, AdminController

Diagram 3 — Use Case Diagram

How to draw in draw.io

Draw a large **system boundary rectangle** labelled "Taxi Meter Backend API". Place **actor stick figures** outside it. Use **ovals** for use cases. Draw **solid lines** from actors to their use cases. Group related use cases in labelled sub-boxes inside the boundary.

Actors (outside the system box)

Actor	Description
Customer (Passenger)	Anonymous — no login required
Driver	Registers and logs in via email or phone OTP
Regulator (Admin)	Logs in via email, manages the platform
Firebase	External system that handles phone OTP verification

Use Cases grouped by feature

Authentication — UC1–UC3

- UC1: Register with email & password ← Driver
- UC2: Login with email & password ← Driver, Regulator
- UC3: Login with phone OTP ← Driver (Firebase is a secondary actor here)

Driver Profile & Identity — UC4–UC7 ← Driver only

- UC4: View own profile
- UC5: Generate QR code
- UC6: Update live location
- UC7: Update fare rate per km

Discovery (public — no login) — UC8–UC10 ← Customer only

- UC8: Browse nearby verified drivers
- UC9: Lookup driver by QR token
- UC10: Get area fare rates

Ride Requests — UC11–UC14

- UC11: Submit ride request with optional fare negotiation ← Customer
- UC12: Poll request status ← Customer

- UC13: View incoming requests ← Driver
- UC14: Accept / reject request ← Driver

Trip Management — UC15–UC18 ← Driver only

- UC15: Start a trip
- UC16: End a trip & auto-generate receipt
- UC17: View my trip history
- UC18: View income summary

Admin / Regulation — UC19–UC22 ← Regulator only

- UC19: View all drivers (with verified/unverified filter)
- UC20: Verify or suspend a driver
- UC21: View all trips system-wide
- UC22: View system stats dashboard

Notable relationship notes

- UC14 (Accept request) **extends** UC15 (Start trip) — accepting a ride request automatically creates a trip
- UC16 (End trip) **includes** auto-generating a receipt — draw a dashed «include» arrow from UC16 to a "Generate Receipt" oval